

Interfaces de comunicación en sistemas embebidos

Comunicación inalámbrica: Bluetooth

Juan José Londoño, M.Sc.

Introducción

Hasta ahora hemos trabajado con protocolos como UART, SPI e I2C, en todos estos casos, los datos viajan a través de conexiones físicas (cables). En estos protocolos, el microcontrolador transmite información cambiando niveles de voltaje en sus pines, representando bits digitales (0 y 1).

En la comunicación inalámbrica, el principio sigue siendo el mismo, seguimos transmitiendo información digital, pero el medio cambia completamente.

En lugar de usar cables, la información se transmite mediante **ondas electromagnéticas en el aire**.

Introducción

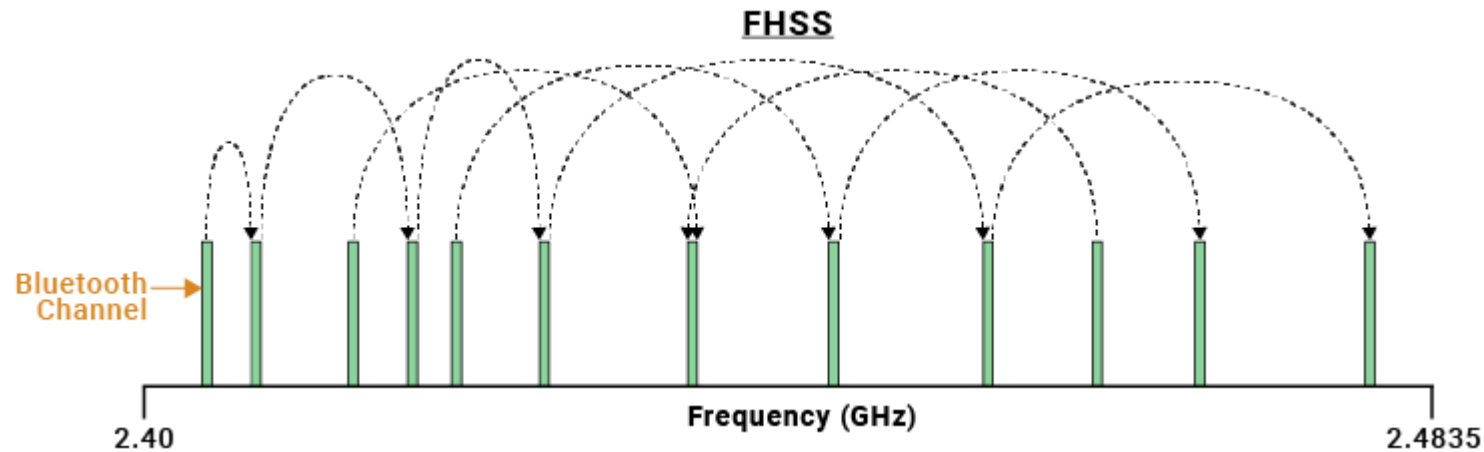
Bluetooth es un estándar de comunicación inalámbrica de corto alcance que permite conectar dispositivos sin necesidad de cables.

Opera en la banda de **2.4 GHz**, una banda de uso libre ampliamente utilizada por distintas tecnologías.

Bluetooth fue desarrollado en los años 90 por la empresa **Ericsson** en Suecia. En 1998 se creó el grupo encargado de su estandarización: **Bluetooth Special Interest Group** (Bluetooth SIG). Este grupo reúne múltiples empresas para asegurar compatibilidad entre dispositivos.

¿Cómo funciona Bluetooth?

Bluetooth transmite datos mediante señales de radio, pero no lo hace usando una sola frecuencia fija, utiliza una técnica llamada **Frequency Hopping** (salto de frecuencia).



Tipos de Bluetooth

Bluetooth no es una sola tecnología. Existen dos principales:

Bluetooth Classic:

Diseñado para transmisión continua de datos, por lo que tiene un mayor consumo de energía. Ejemplo: audio (audífonos, parlantes).

Bluetooth Low Energy (BLE):

Diseñado para bajo consumo ideal para sensores y sistemas embebidos. Muy usado en dispositivos portátiles e IoT.

Bluetooth classic

También conocido técnicamente como **Bluetooth BR/EDR**.

Bluetooth Classic fue la primera forma de Bluetooth ampliamente utilizada y está orientada principalmente a aplicaciones donde se necesita una **conexión continua** entre dispositivos.



Stack Bluetooth

Bluetooth classic no es solo mandar datos por radio, es una pila de protocolos organizada por capas, esto se conoce como pila Bluetooth.

Las capas se pueden dividir de forma simplificada en:

- Aplicación
- Perfil Bluetooth
- Protocolos superiores
- Controlador Bluetooth
- Radiofrecuencia

Stack Bluetooth

Radiofrecuencia:

Es la capa física. Aquí se define cómo se transmite la información por el aire usando ondas electromagnéticas.

Baseband:

La baseband se encarga de organizar la comunicación básica entre dispositivos. En Bluetooth Classic, los dispositivos se organizan en una pequeña red llamada **piconet**. Una piconet tiene un dispositivo que coordina la comunicación y otros dispositivos conectados a él.

Stack Bluetooth

Link Manager Protocol, LMP:

El LMP se encarga de administrar el enlace entre dispositivos, es decir, mientras la *baseband* se preocupa por la transmisión básica, el LMP se preocupa por la gestión lógica de la conexión.

Sus funciones incluyen:

- Establecer conexión
- Configurar parámetros del enlace
- Negociar características soportadas

Stack Bluetooth

HCI: Host Controller Interface

HCI es la interfaz entre dos grandes partes del sistema Bluetooth, el **host** y el **controller**.

El controller maneja la parte de radio, *baseband* y control del enlace.

El host maneja protocolos superiores y perfiles de aplicación.

Stack Bluetooth

L2CAP (Logical Link Control and Adaptation Protocol):

Su función es adaptar los datos de las capas superiores para que puedan enviarse por el enlace Bluetooth. L2CAP permite que varios servicios usen la misma conexión Bluetooth sin mezclarse entre sí.

Perfiles Bluetooth:

Los perfiles definen cómo se usa Bluetooth para una **aplicación concreta**.

Perfiles Bluetooth

Bluetooth no define directamente cómo usar los datos en cada aplicación, para eso existen los perfiles Bluetooth.

El perfil dice “cómo” usar Bluetooth para una aplicación concreta.

Dos dispositivos pueden soportar Bluetooth, pero si no soportan el mismo perfil, **no pueden comunicarse** correctamente.

Perfiles Bluetooth

- **A2DP (Advanced Audio Distribution Profile):** Transmisión de audio estéreo. Usado en audífonos y parlantes Bluetooth.
- **HFP (Hands-Free Profile):** Comunicación de voz. Usado en carros (manos libres) y headsets.
- **HID (Human Interface Device):** Dispositivos de entrada. Usado en teclados, mouse y controles.
- **OBEX / FTP:** Transferencia de archivos. Usado en envío de fotos e intercambio de archivos.

SPP (Serial Port Profile)

Este perfil permite hacer algo muy poderoso, convertir Bluetooth en un “UART inalámbrico”.

SPP es un perfil que utiliza el protocolo RFCOMM para crear un canal de comunicación serial virtual entre dos dispositivos.

Aunque SPP “se ve” como UART, internamente es mucho más complejo.

Descubrimiento de dispositivos (Inquiry)

Antes de que dos dispositivos Bluetooth puedan comunicarse, primero deben poder encontrarse. En Bluetooth Classic, este proceso se llama **inquiry**.

Un dispositivo entra en modo de búsqueda y comienza a escanear el entorno en busca de otros dispositivos cercanos que estén en modo visible.

Como resultado de este proceso, obtiene información básica como la dirección del dispositivo (BD_ADDR) y, en muchos casos, su nombre.

Emparejamiento (Pairing)

Una vez que un dispositivo ha encontrado a otro, el siguiente paso es establecer una relación de confianza, lo cual se logra mediante el proceso de emparejamiento.

Durante este proceso, los dispositivos intercambian información para autenticarse entre sí.

El objetivo del pairing es asegurar que ambos dispositivos están autorizados para comunicarse y generar claves que podrán reutilizarse en futuras conexiones sin necesidad de repetir el proceso.

Conexión (ACL Link)

Una vez que los dispositivos se han emparejado, se establece un enlace de comunicación llamado ACL (*Asynchronous Connection-Less*).

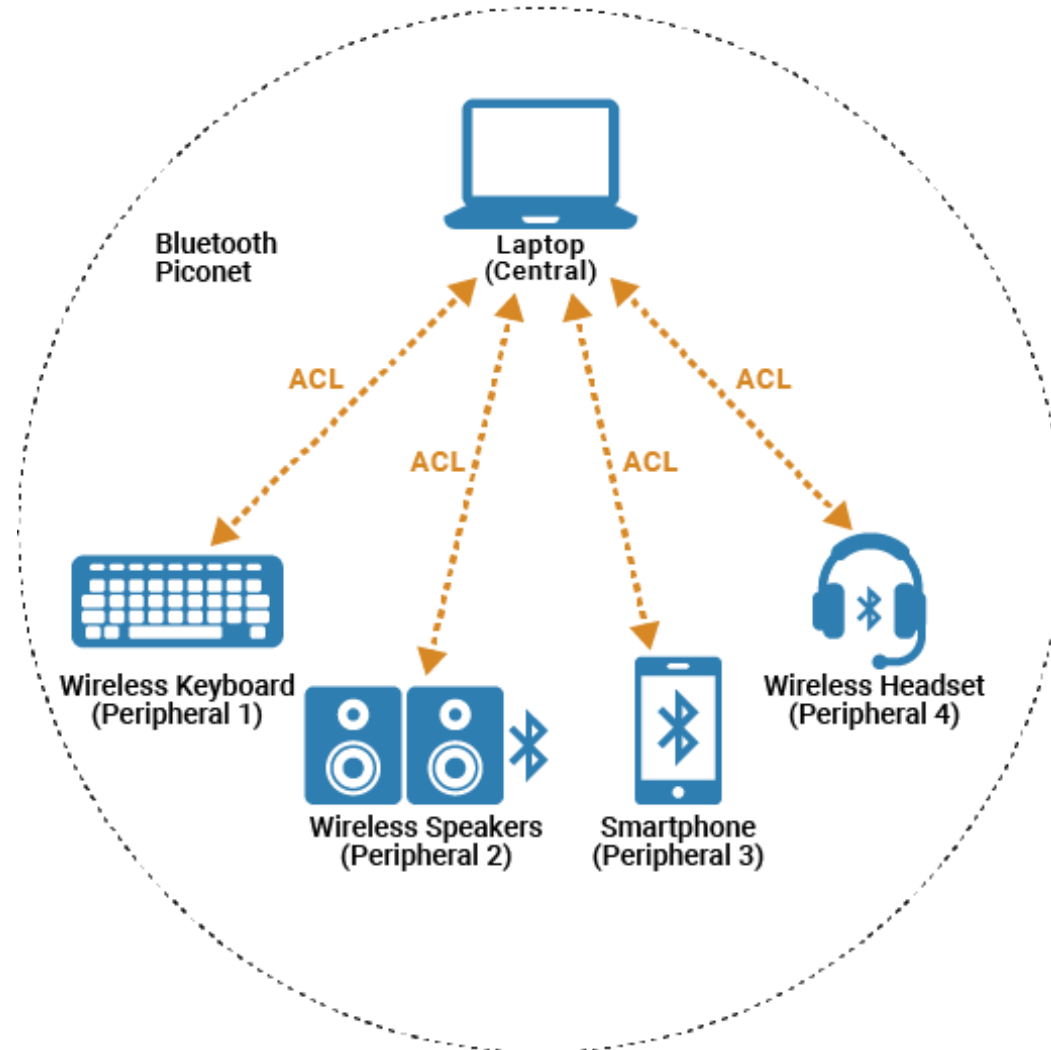
Este es el tipo de conexión principal utilizado para transmitir datos en Bluetooth Classic.

El enlace ACL permite enviar paquetes de datos de forma bidireccional entre dispositivos, manejando aspectos como retransmisión en caso de errores y control de flujo. Es, en esencia, el canal sobre el cual operan los perfiles de datos.

Roles y piconet

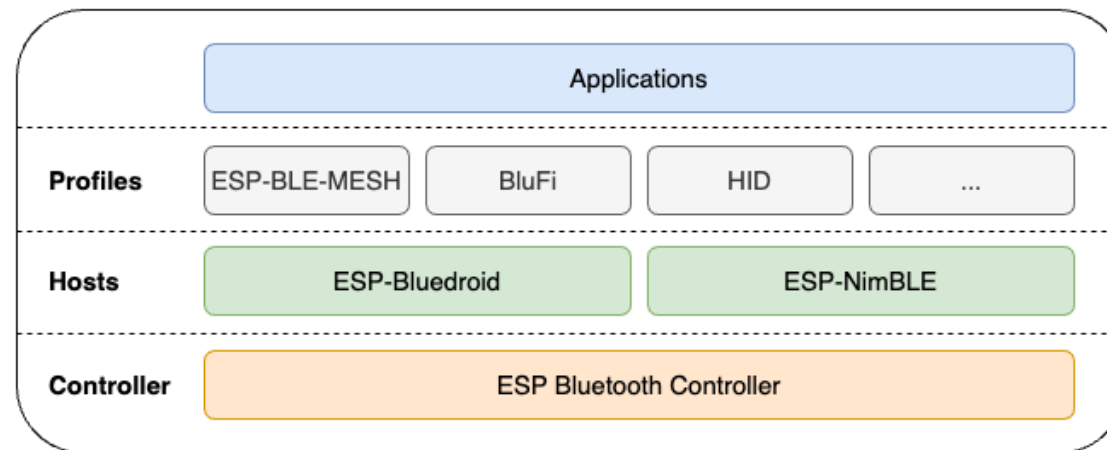
En Bluetooth classic, la comunicación está organizada bajo una estructura jerárquica donde un dispositivo actúa como maestro y uno o más dispositivos actúan como esclavos.

El conjunto formado por un maestro y los dispositivos esclavos conectados a él se denomina **piconet**. En una piconet, el maestro puede tener hasta **siete esclavos activos simultáneamente**, además de otros en estado inactivo. Todos los dispositivos dentro de la piconet comparten un mismo esquema de sincronización y salto de frecuencia, lo que permite que la comunicación sea ordenada y eficiente.



Bluetooth en ESP32

El ESP32 tiene soporte integrado para Bluetooth tanto en su versión classic (BR/EDR) como en BLE. Esto significa que no necesitamos hardware adicional para implementar comunicación Bluetooth; todo el stack (pila) está incluido en el chip.



Modelo basado en eventos

El manejo de Bluetooth en ESP-IDF es event-driven, esto significa que el sistema genera eventos como:

- Dispositivo conectado.
- Dispositivo desconectado.
- Datos recibidos.
- SPP iniciado.

Y nosotros debemos implementar funciones (***callbacks***) que respondan a esos eventos.

Flujo de configuración

Cuando se trabaja con Bluetooth Classic en ESP-IDF, el flujo general es:

- Inicializar el controlador Bluetooth.
- Inicializar el stack Bluetooth.
- Habilitar modo classic e Inicializar SPP.
- Registrar callbacks.
- Esperar conexión.
- Enviar/recibir datos.

Configuración de dependencias

Si se trabaja con PlatformIO, es necesario habilitar las dependencias de Bluetooth classic para habilitar todo el stack, ya que, por defecto, estas no están incluidas en el proyecto.

Para ello cree en la raíz del proyecto un archivo llamado:

```
sdkconfig.defaults
```

Configuración de dependencias

En el archivo creado, pegue el siguiente texto:

```
CONFIG_BT_ENABLED=y  
CONFIG_BTDM_CTRL_MODE_BLE_ONLY=n  
CONFIG_BTDM_CTRL_MODE_BR_EDR_ONLY=y  
CONFIG_BTDM_CTRL_MODE_BTDM=n  
CONFIG_BT_CLASSIC_ENABLED=y  
CONFIG_BT_SPP_ENABLED=y  
CONFIG_BT_BLE_ENABLED=n
```


Configuración de dependencias

Luego, en el archivo **src/CMakeLists.txt**, reemplace el contenido por el siguiente:

```
idf_component_register(  
    SRCS "main.c"  
    INCLUDE_DIRS "."  
    REQUIRES bt nvs_flash driver  
)
```

